

L06911 · 鸿蒙操作系统 (2026 秋)

第 1 章智能终端操作系统概述

课程导学·操作系统关键价值·智能终端 OS 演进趋势

江西科技师范大学

万博洋

2026 年 9 月 2 日 (第 1 教学周)

本讲目录

1. 课程导学
2. 操作系统架构和关键价值
3. 系统调用初识：应用如何使用 OS
4. 通用操作系统简史
5. 智能终端操作系统简史
6. 智能终端操作系统演进趋势与挑战
7. 小结与延伸

1. 版权与来源说明

使用声明

本课件仅供教学使用，以《智能终端操作系统与 OpenHarmony 实践》（陈海波等，电子工业出版社，2024，ISBN 978-7-121-48972-3）为基准编写。

- 教材/外部插图按图号与来源标注；本讲当前图均为课程自制，后续引用外部图片将在图注标注来源与许可
- 代码示例为示意代码，非鸿蒙源码；学术数据注明出处
- 课件编写与校核使用 AI 辅助并经人工核校；学生使用 AI 需按要求声明；本课件仅供教学使用

1. 授课教师

万博洋 (Boyang Wan)

江西科技师范大学·信息工程学院·讲师

- 研究团队：人工智能与生物信息认知 (AIBC) 研究组
- 研究方向：视觉理解、图像描述、AI for Science
- 学术成果：发表论文十余篇，被引超过 1000 次
- 个人主页：<https://wanboyang.github.io/>

1. 课程介绍



新强工业化·特色化示范性高质量软件人才培养系列
(OpenHarmony布道师)

智能终端操作系统 OpenHarmony实践

陈海波 夏度诚 杜冬冬 吴明喻 古金宇 / 编著



电子工业出版社

Publishing House of Electronics Industry

北京·BEIJING



电子工业出版社

PUBLISHING HOUSE OF ELECTRONICS INDUSTRY

- 课程名称：鸿蒙操作系统（L06911） | 3 学分 | 48 学时
- 适用班级：电子信息工程专业 25 级（创新班）1 班
- 主教材：《智能终端操作系统与 OpenHarmony 实践》（电子工业出版社，2024）

ISBN 978-7-121-48972-3

1. 课程介绍 (续)

- 教学安排：14 个教学周（第 1–6、9–16 周），每周三；第 7–8 周线上项目任务
- 考核方式：考试课程（期末闭卷 100 分钟）；平时 30%（含课程项目）+ 期末考试 70%

课程网站: <https://wanboyang.github.io/zh/harmonyos/>

教学总览: <https://wanboyang.github.io/zh/teaching/>

课程网站预览

Wanboyang Wan
(万博杨)

HarmonyOS and
OpenHarmony
University

课程介绍

课程简介

课程大纲

课程资源

课程项目

课程问答

课程反馈

课程介绍

课程简介

课程大纲

课程资源

课程项目

课程问答

课程反馈

课程介绍

课程简介

课程大纲

课程资源

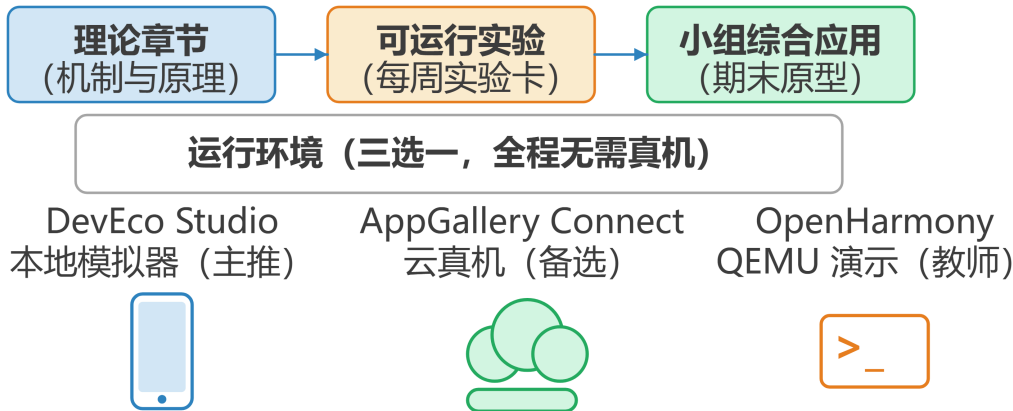
课程项目

课程问答

课程反馈

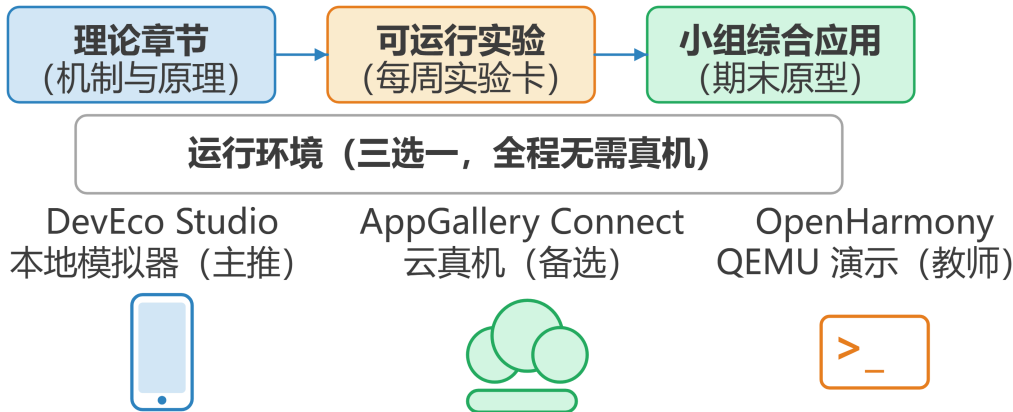
1. 课程项目（做中学）·理论章节

每周实验卡对应课堂机制与原理：本周（第 1 讲）建立“OS 是什么”的直觉，下节进入 Hello World 与系统调用。



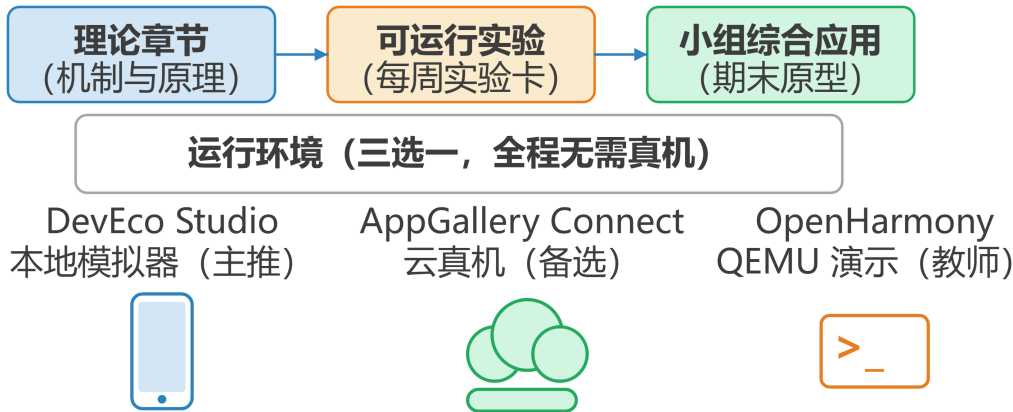
1. 课程项目（做中学）·可运行实验

每周实验卡 30–60 分钟，与理论同步推进：跑通一次能看见“OS 做了什么”的实验（编译、运行、观察）。



1. 课程项目（做中学）·小组综合应用

3-4 人小组，期末完成一个可运行的原型：从“调用 API”走向“理解并改进系统”。



1. 课程项目（做中学）·零真机方案

本地模拟器（主推） | AppGallery Connect 云真机（备选） | 教师

OpenHarmony QEMU 演示，全程无需真机。

理论章节
(机制与原理)

可运行实验
(每周实验卡)

小组综合应用
(期末原型)

运行环境（三选一，全程无需真机）

DevEco Studio
本地模拟器（主推）



AppGallery Connect
云真机（备选）

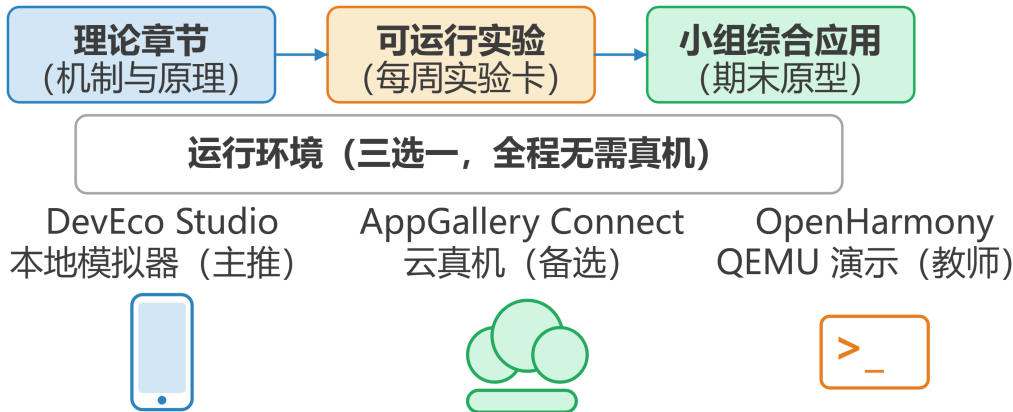


OpenHarmony
QEMU 演示（教师）



1. 课程项目（做中学）·选题方向

课程学习助手、校园记账本、健康运动助手、AI 问答助手、智能家居控制
(可自定义)。



1. 课程项目（做中学）·关键节点

第 7–8 周开题 + 原型 Demo；第 16 周答辩。评分：开题 2 + 里程碑 3 + 答辩 5 = 10 分；防抄袭：starter code + 代码走查 + 答辩，AI 使用需声明。



运行环境（三选一，全程无需真机）

DevEco Studio
本地模拟器（主推）



AppGallery Connect
云真机（备选）



OpenHarmony
QEMU 演示（教师）

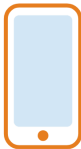


1. 课程项目路线图

表: 课程项目路线图 (与每周理论同步推进)

阶段	周次	里程碑	对应章节
环境与骨架	第 2 周	DevEco Studio + 模拟器跑通 Hello 应用	第 2 章四层架构
内核机制	第 3-6 周	生命周期/后台调度/内存观察/数据持久化	第 3-6 章
原型里程碑	第 7-8 周	开题报告 + ≥ 3 页面可运行 Demo	第 1-6 章
系统服务	第 9-13 周	Want/IPC、动画音频、网络传感、分布式	第 7-15 章
框架与交付	第 14-16 周	Stage 重构、性能优化、安全加固、答辩	第 16-22 章

1. 课程导学：从功能机到万物智联



智能终端 OS 演进：
从硬件附属 → 全场景底座

1. 为什么学习操作系统

系统领域是“黑土地”

Windows、Linux、macOS、Android、iOS 不是终点：汽车（QNX）、工业（VxWorks）、云端（Linux）都在按场景选 OS；设备形态越多，OS 形态越多。

- 系统领域是软件产业基石：微软、IBM、华为、阿里都做系统
- 图灵奖与 OS 同行：Brooks (1999)、Corbató (1990)、Ritchie (1983)
- 学好 OS：更快找到 bug、理解性能、能构建自己的系统

1. 这门课将带给你什么能力

- 成为更高效的程序员：快速定位 bug、解释并调试性能
- 理解复杂系统的设计与权衡：抽象、资源、隔离
- 构建能力：从“调用 API”到“理解并改进系统”
- AI 编程时代：能判断 AI 生成的系统代码是否合理

1. 操作系统难，但值得

“把复杂留给自己，把简单留给用户”

直接管理硬件细节，需要理解寄存器、中断、内存等；但正因如此，才能从根本上高效使用资源。

- 抽象与具体：抽象帮助理解，bug 逼你进入实现细节
- “What I cannot create, I do not understand.” ——Richard Feynman
- 本课程用“做中学”降低门槛：每周实验卡 + 小组应用

1. 本讲目标与三问

学完本讲，你能

- 一句话说明 OS 的“承上启下”
- 用一个 Hello World 说出 OS 至少 4 项工作
- 解释 `printf`→`write`→`sys_write` 并用 `strace` 实证
- 举出“流氓程序”案例，并说冲突与解决思路

本讲三问

- 为什么说 OS 是“计算产业的基础与核心”？
- 功能机到万物智联，成功要素如何变化？
- 五大趋势怎样指向“统一底座 + 多端协同 + 智能融合”？

1. 思考：如果没有操作系统，应用开发会面临什么？

先想一想

先独立想一想：没有 OS，一个手机 App 还要自己处理哪些事？

2. 定义：操作系统是什么

应用软件（浏览器、播放器、App...）



操作系统

OS

管理硬件资源 · 控制程序运行 ·
改善人机交互 · 提供运行环境

硬件（CPU · 内存 · 存储 · 设备）

2. 例子：从 Hello World 说起

```
#include <stdio.h>

int main() {
    printf("Hello
    ↪ World!\n");
    return 0;
}
```

```
$ gcc hello.c -o hello
$ ./hello & ./hello
[1] 144
Hello World!
Hello World!
[1]+  Done  ./hello
```

示例素材：上海交大 IPADS《操作系统》导论 (CC BY 4.0)。示例输出顺序可能因进程调度不同而异。

2. 运行 hello 时，OS 要回答什么？

先看现象，再想实现

- 可执行文件存储在哪里、如何被加载到 CPU？
- 如何把 “Hello World!” 输出到屏幕？
- 两个 hello 为什么能同时运行？
- 谁管理 CPU、内存、文件与设备？

这些问题正是后续各讲将逐一展开的主线。

2. 关键价值：把硬件能力转化为应用能力

没有操作系统
每个应用自管
CPU/内存/设备
开发门槛极高

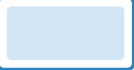


有操作系统
统一接口
硬件能力 → 应用能力



2. 现代操作系统：内核到应用框架

应用与服务：浏览器 · App · 业务服务



应用框架：组件、UI、应用管理



操作系统服务：基础服务、分布式服务、软总线



内核与 Shell：进程、内存、文件、设备、命令接口



承上：运行时、开发环境、生态入口 | 启下：抽象并协同硬件

2. 产业演进：OS 定位不断变化

主机时代
OS 是硬件附属

PC 时代
独立软件产品

移动互联网
云服务入口

万物智联
统一底座 +
多端协同



2. 成功要素随产业演进

OS

操作系统（一次开发·多硬件运行）

主机时代
性能

PC 时代
通用性

移动互联网
生态/黏性

万物智联
智能化与赋能

新旧模式长期并存：设备竞争力、生态入口、
行业赋能

3. 系统调用链：应用如何请求 OS



系统调用 = 应用请求操作系统服务的机制
像普通函数调用一样，但会陷入内核

strace: `printf` → `write` → `sys_write`, 能看到应用与内核的每一次交互

3. 用 strace 看见系统调用

```
$ strace ./hello
execve("./hello", ["./hello"],
↪ ...) = 0
...
write(1, "Hello World!\n", 13) =
↪ 13
exit_group(0) = ?
```

课堂观察

应用认为自己只是在“打印一行字”；操作系统看到的是一系列系统调用。

- printf 只是库函数；进入内核的是 write
- 1 表示标准输出；13 表示写入 13 个字节
- 程序退出前还会调用 exit_group
- strace 是只读的观察工具，不修改程序

3. 系统调用：应用与操作系统的边界

应用能做什么、不能做什么？

应用可以直接访问 CPU、内存吗？大部分不能；操作系统通过系统调用提供安全、受控的能力。

- 服务：CPU/内存/文件/网络抽象，进程间通信与同步原语
- 管理：生命周期、资源分配、安全与隔离
- 冲突：单应用效率 vs 系统整体利用率（如频繁切换进程带来的调度开销）
- 追问：只有一个专用程序，还需要 OS 吗？（第 3 周回看）

3. OS 的管理（一）：不让一个程序独占 CPU

```
/* rogue.c: 一个“流氓程序” */  
int main() {  
    while (1);  
}
```

OS 如何解决？

- 现象：程序会一直占用 CPU，其他任务无法运行
- 方法 1：每 10 ms 产生时钟中断（时间片）
- 调度器决定下一个要运行的任务
- 方法 2：通过信号打断当前任务
- 例如：kill -9 <PID>

3. OS 的管理 (二): 不让一个程序拖垮系统

```
/* rogue-1.c: 不断创建新进程 */  
int main() {  
    while (1)  
        fork();  
}
```

OS 如何解决?

- 现象: 不断 fork 会耗尽进程表与内存
- 资源配额: cgroup / Linux
- 虚拟化: 虚拟机限制资源
- 制度约束: 应用商店预审准入
- 兜底手段: 杀死进程或重启系统

课堂演示提醒: fork() 炸弹会快速耗尽系统资源, 只在教师受控虚拟环境演示, 不要在学生机或共享机运行。

4. 通用操作系统发展时间线



动力：效率 → 可编程 → 交互 → 易用 → 生态 → 场景

课程自制图

4. 操作系统史 (1/2): 通用 OS 的里程碑

系统与人物	年份	关键贡献
GM-NAA I/O	1956	批处理雏形
OS/360	1964	首个通用 OS; 软硬件解耦
MULTICS (Corbató)	1960s (1965–1969)	分时/文件系统/动态链接
UNIX (Thompson/Ritchie)	1969	简洁设计、Shell; 催生 C

资料整理自教材第 1 章与 IPADS 《操作系统》导论。

4. 操作系统史 (2/2): 开放、界面与移动时代

系统与人物	年份	关键贡献
Linux (Torvalds)	1991	开源、服务器与云计算的主流底座
Xerox → Mac → Windows	1973–1985	图形界面与个人计算普及 (Macintosh 1984、Windows 1.0 1985)
iOS / Android / OpenHarmony	2007–2020s	移动互联网与万物智联的新路线

资料整理自教材第 1 章与上海交大 IPADS《操作系统》导论。

4. AI 会错：一起验证

课堂任务（2 分钟）

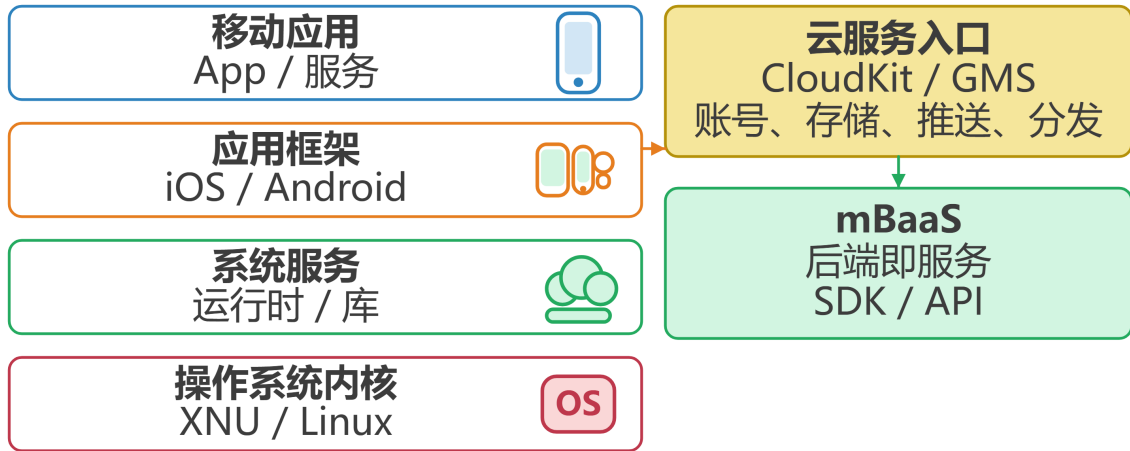
让 AI 生成“操作系统发展时间线”初稿，对照教材图 1-2/图 1-7 找到至少一处事实错误或过度简化。

- 记录方法：AI 说了什么 → 我用什么证据验证 → 哪里纠正了 AI
- 演示方式：教师现场提问/投影，或使用课前准备的 AI 回答截图
- 本期原则：AI 能生成看似可信的系统知识；“验证优于信任”从第 1 讲开始

5. 智能终端 OS 三时代



5. 智能机通用架构与云服务入口



5. 智能终端 OS 三时代 (续)



5. 万物智联四特征

泛在化

设备与连接数激增
人机物泛在融合



场景化

一对一→多对多
关注场景整体体验



智能化

感知/互联/数据驱动
自适应与个性化



生态化

设备/应用/服务协作
互联互通、共享协同



5. 元 OS 架构：One OS Kit for All

设备多样性与生态统一性的矛盾：昆虫纲悖论

哺乳动物约 6000 种（服务器/PC）
昆虫约 100 多万种（多样化终端）
种类多、差异大 → 难形成规模效应

组件化解耦与按需合成
兼顾统一维护与弹性组合

元OS架构：One OS Kit for All
组件化解耦 · 按需合成 · 场景定制

5. 课堂思考：为什么说智能终端操作系统是“全场景智能底座”，而不仅是手机系统？

先独立思考，再对照解答

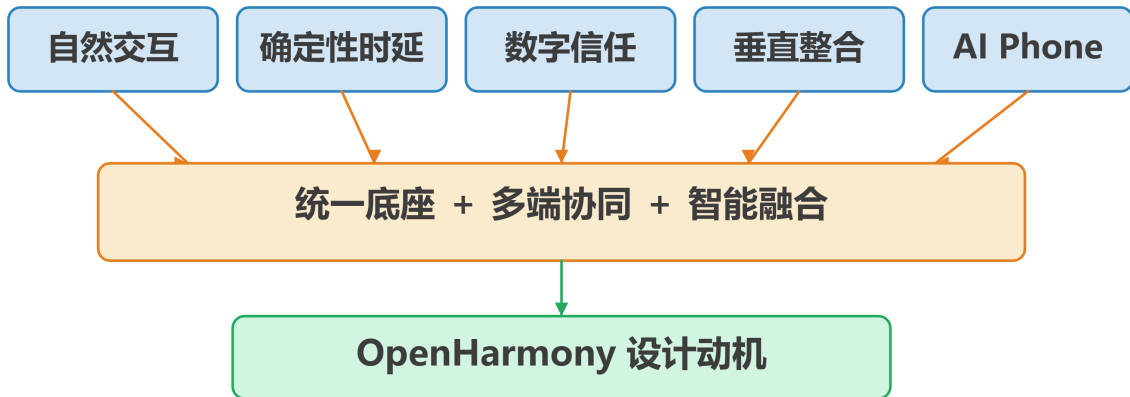
结合本节讲过的机制与代价来推理。

5. 解答：为什么说智能终端操作系统是“全场景智能底座”，而不仅是手机系统？

参考解答

- 终端形态多样：手机、平板、手表、车机、家居等都需要操作系统支撑
- 单一系统难以覆盖所有场景，需要统一底座与多端适配
- 向上支撑消费与行业应用，向下赋能半导体等产业链

6. 五大演进趋势



6. 课堂思考：五大趋势中，哪一条最能解释 OpenHarmony 的诞生？

先独立思考，再对照解答

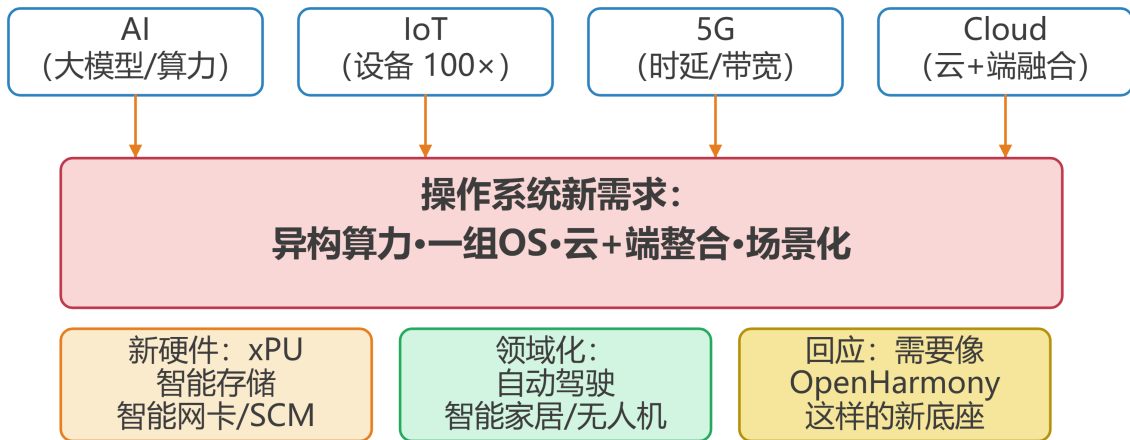
结合本节讲过的机制与代价来推理。

6. 解答：五大趋势中，哪一条最能解释 OpenHarmony 的诞生？

参考解答

- 万物智联设备数量与形态激增 → 需要“统一底座 + 多端协同”（教材 1.2.3 的泛在 OS、元 OS 架构）
- 五趋势说明：未来还需跨层协同、系统级智能与可信基础
- OpenHarmony 是这些思想的工程实践之一，第 2 讲展开

6. AIoT: 新硬件与新场景



6. AIoT 不是抽象词：把趋势落到数字与案例

三组数量级

- 5G：吞吐与时延各改善 10–100 倍
- AI 硬件计算能力需求约提升 100 倍
- IoT：连接数量约提升 100 倍

四个产业案例

- 2018 年 Android 对欧盟设备收费，最高约 40 美元/台
- 2018 年 IBM 340 亿美元收购 Red Hat
- 2016 年起 Google 600+ 人力研发 Fuchsia
- Linux 超 3000 万行，约 200 万行/年更新

来源：IPADS OS-01 (CC BY 4.0)。

6. 一个“预测翻车”：从专用到通用，再到专用

1943 年的著名判断

“I think there is a world market for maybe five computers.” ——Thomas Watson, IBM 总裁, 1943

来源：IPADS OS-01 (CC BY 4.0)。

- 现实：计算机与 OS 种类反而爆发式增长
- 专用 → 通用 → 专用：汽车等场景需要更专用的 OS
- 云 + 端垂直整合：从分层解耦走向跨层协同
- 结论：未来是“更多系统在更多场景协同”

6. 回到 OpenHarmony：为什么需要新 OS

从“趋势”到“答案”

新场景需要统一底座 + 多端协同 + 智能融合；设备从 KB 级到 GB 级、场景各异，需要组件化解耦与按需合成；OpenHarmony 以通用微内核 + One OS Kit 思想回应（教材 1.2.3 / 1.3.4）。

- 生态控制：OS 不是免费午餐，而是“黑土地”
- 领域化：专用 → 通用 → 专用，OS 种类会更多
- 本课程主线：理解这套“新 OS”的机制，并做出可运行应用

7. Take-away (1/2): 本讲主线

核心要点

- 操作系统四大功能：资源管理、程序运行、人机交互、应用支撑
- 例子：Hello World 背后是加载、输出、并发与资源管理
- “流氓程序”说明 OS 必须同时提供管理与服务
- 系统调用是可观察的：printf→write→sys_write, strace 可见

7. Take-away (2/2): 演进与考点

核心要点

- 三阶段：功能机 → 智能机 → 万物智联（泛在 OS、元 OS 架构）
- 五大趋势：自然交互、确定性时延、数字信任、垂直整合、AI Phone
- AIoT：5G/AI/IoT 数量级、Android 收费、Red Hat、Fuchsia
- 考点：代表系统、成功要素

7. 鸿蒙人才生态与华为认证：这门课通往哪里

政策与产业

2025-04 中共中央：AI 全学段通识；2026-01 工信部鸿蒙人才促进工作组（130+ 单位）。

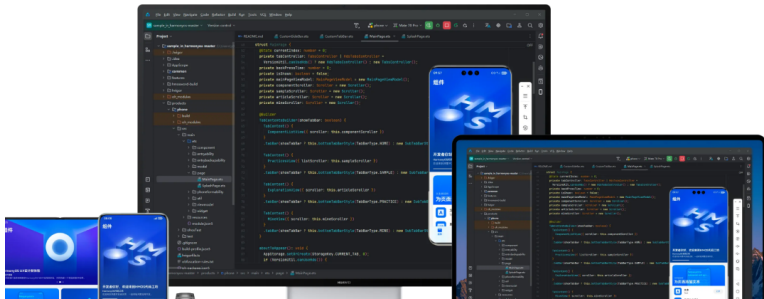
- **认证三级**：HCIA→HCIP→HCIE；HarmonyOS 为应用开发方向
- **规模**：认证人才 110 万 +、HCIE 4 万 +
- **一试双证**：华为证书 + 职业技能等级证书（移动操作系统应用设计员）
- **本课衔接**：为 HCIA-HarmonyOS 打基础

7. 延伸资源与实践

课后延伸

- **《鸿蒙生态解决方案白皮书》第 1 章：生态规模、智能终端趋势**
(课堂发放)
- **开发者学堂《HarmonyOS 第一课》：视频导学：**
<https://developer.huawei.com/consumer/cn/training/>
- **暑期预习方案（视频学习）：**课堂发放
- **上海交大 IPADS《操作系统》课程（同作者：陈海波/夏虞斌，含 ChCore 实验）：** <https://ipads.se.sjtu.edu.cn/courses/os/>

7. 开发环境：DevEco Studio



DevEco Studio——HarmonyOS 应用与元服务的集成开发环境（IDE）：AI 辅助编程、编译构建、UI 实时预览、代码调试与模拟器。

来源：华为开发者联盟官网。

7. DevEco 工具链：Studio • CLI • Code

工具	定位	交互	核心功能 / 主要用户
DevEco Studio	集成开发环境 (IDE)	GUI	编辑、调试、UI 实时预览、性能分析；面向开发者（人）
DevEco CLI	命令行工具集	CLI	hvmorw 构建、ohpm 包管理、hdc 连接；面向 CI/CD 与 AI Agent
DevEco Code	AI 编程助手 (AI Agent)	终端对话	自然语言 → 自动编码、修 bug、跑通构建；与 AI 结对开发

三者如何协作？

- Studio 提供能力（基石）；CLI 封装为可脚本调用的原子命令（桥梁），服务 CI/CD 与 AI Agent
- Code 用自然语言指挥 CLI/Studio；需先装好 Studio，与内置 CodeGenie 是独立产品

选择：日常开发 → Studio；自动化构建 → CLI；AI 结对协作 → Code。

7. 实验卡 (1/2): 安装与创建

目标 (30–60 分钟)

安装 DevEco Studio, 新建空应用, 在模拟器或云真机中跑通并提交截图/日志。

1. 下载并安装 DevEco Studio (Windows 10/11, 建议 16 GB 内存, 开启虚拟化)
2. 配置 HarmonyOS SDK (建议使用 API 14–22 中的可用版本), 创建 Empty Ability 空应用
 - 本周不要求理解代码, 关键是先建立“能运行鸿蒙应用”的起点

7. 实验卡 (2/2): 运行与提交

1. 优先启动本地模拟器; 装不上时申请 AppGallery Connect 云真机, 课程另备 OpenHarmony QEMU 演示
2. 跑通空白页面, 截图 “应用界面 + DevEco 版本 + 模拟器状态”, 连同安装日志提交课程平台
 - 遇到问题先记录报错原文, 再按关键词检索; 仍卡住则到课堂/答疑群处理

7. 作业与预习

课后任务

- 完成实验卡：安装 DevEco Studio + 空应用 + 截图/日志
- 绘制智能终端 OS 演进时间线
- 可选观察：用 strace 查看一个 hello 程序产生的系统调用并截图
- 预习第 2 章，思考 OpenHarmony 为何面向万物智联重新设计
- AI 验证：让 AI 生成“OS 发展史”并对照教材找 1 处错误

7. 下次课预告

预告

第 2 讲：OpenHarmony 概述——四层技术架构与行业案例